Handout #10
Thursday, 1 October 2026

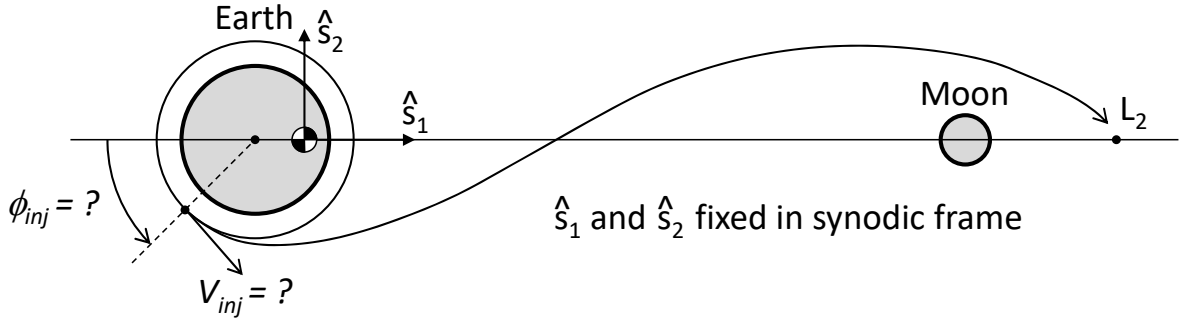
Stanford AA 279B
Advanced
Space Mechanics
Prof. Barrows

Problem Set 2

Autumn Quarter 2026

Due: Thursday, 8 October 2026 at 1:30pm via Gradescope
Topics: Three-body problem targeting using shooting method, halo orbits, relative motion, Clohessy-Wiltshire equations, basic optimization

2.1) (28 points) As on Problem Set 1 Problem 2, here you'll consider trajectories of a spacecraft moving in the Earth-Moon system. But this time, rather than being given starting values of v_{inj} and φ_{inj} , *you* will determine the initial conditions on v_{inj} and φ_{inj} . The objective is to leave LEO with v_{inj} and φ_{inj} and end up at the Earth-Moon L_2 point 4.0 [days] after leaving LEO. Once at L_2, there will be a burn of magnitude ΔV_{L2} to remove all velocity (relative to the synodic frame).



a) Find the synodic coordinates of L_2 of the Earth-Moon system. Use $\mu_{Earth} = 398{,}600$ [km^3/sec^2] , $\mu_{Moon} = 4{,}903$ [km^3/sec^2] , an Earth-Moon distance of ${}^{Earth}r^{Moon} = 384{,}400$ [km] , and a LEO altitude of 200 [km]. Mean equatorial radii are $r_{Earth} = 6{,}378.0$ [km] and $r_{Moon} = 1{,}738.0$ [km] .

b) Use a shooting method to iterate to the values of v_{inj} and φ_{inj} that cause the spacecraft to end up at L_2 after exactly 4.0 [days]. Consider your answer close enough when the spacecraft finishes within 1.0 [km] of L_2. Note that v_{inj} and φ_{inj} (and not x- and y-velocities as in the lecture notes example) must be varied to form the Jacobian. Each time v_{inj} or φ_{inj} changes, the starting values of x and y change as well. You will need to

determine starting guesses and suitable sizes for your variations in v_{inj} and φ_{inj} in order to get convergence. What values of v_{inj} and φ_{inj} did you find?

c) Plot the resulting trajectory in synodic coordinates from LEO to L_2 in the x, y plane in the same way you did for Problem Set 1. Show the location of L_2 and also include circles showing the correct locations and radii of Earth and the Moon.

d) What is the scalar value ΔV_{L2} of the retro-burn at L_2?

e) As an approximation of how much propellant is required to reach L_2 from LEO, what is

$$\Delta V_{\text{total}} = \left(v_{inj} - \left[\sqrt{\frac{\mu_{Earth}}{(r_{Earth}+200.0)}} - \omega_s (r_{Earth} + 200.0) \right] \right) + \Delta V_{L2} \quad ?$$

(In case you are curious, by varying the time-of-flight to values other than 4.0 [days], you can minimize ΔV_{total} using a duration between six and seven days.)

2.2) (8 points) Now you'll explore two "halo orbits" near L_1 in the Earth-Moon system using the same physical constants of Problem 2.1.

a) Assume a spacecraft begins with these initial conditions in the synodic reference frame:

$$^{I}\underline{r}_s^3 = [+343777.2401 \quad 0.0 \quad -27449.52657]^{\mathrm{T}}$$
$$^{s}\underline{v}_s^3 = [0.0 \quad -0.2769119497 \quad 0.0]^{\mathrm{T}}$$

Simulate the motion of the spacecraft for 30 [days]. Plot the orbit in the x, y plane along with the location of L_1 and circles representing the Earth and Moon. To within 0.1 [days] what is the "period" of this halo orbit? (I.e. after it leaves its starting point, when does it make its next crossing of the y = 0 plane with its y-velocity having the same sign as its initial y-velocity?)

b) Repeat part (a) for the following different set of initial conditions:

$$^{I}\underline{r}_s^3 = [+343117.419 \quad 0.0 \quad +76029.48372]^{\mathrm{T}}$$
$$^{s}\underline{v}_s^3 = [0.0 \quad +0.2045850249 \quad 0.0]^{\mathrm{T}}$$

Submit your plot and your estimate for the period of this halo orbit.

2.3) (32 points) Here you'll simulate motions of a "chaser" spacecraft moving in a CW frame fixed to a "reference" spacecraft in a 760 [km] altitude circular orbit around the Earth. Assume the Earth is spherical with radius $R_{\text{Earth}} = 6{,}378$ [km] and gravitational parameter $\mu_{\text{Earth}} = 398{,}600\ [\text{km}^3/\text{sec}^2]$. The chaser spacecraft begins at:

$$\begin{bmatrix} x_0 \\ y_0 \\ \dot{x}_{-0} \\ \dot{y}_{-0} \end{bmatrix} = \begin{bmatrix} 25 \\ -75 \\ 0 \\ 0 \end{bmatrix} \begin{matrix} [km] \\ \\ [km/sec] \\ \\ \end{matrix}$$

and is trying to get to the reference spacecraft at the origin.

a) What ΔV (burn at the starting point) is required to perform an intercept at the reference (with no burn to stop at the reference) after 10 [min]?

b) What additional ΔV (burn at the origin) is required to perform a rendezvous (to stop at the reference) after 10 [min]? What is the total ΔV for the rendezvous? Using the exact solution to the CW equations, plot this trajectory with markers every minute to help estimate speed. You can use MATLAB's `set(gca,'XDir','reverse');` command to reverse the direction of the horizontal axis.

c) Now we'll consider transfer time t_f to be a free variable in performing the rendezvous. For values of t_f in 1 [min] increments ranging from 1 [min] to 120 [min], make a plot of total ΔV versus transfer time.

d) What is the optimal value of t_f that minimizes total ΔV? What is the minimum total ΔV? Plot the optimal trajectory with 5 [min] markers along the path.

2.4) (32 points) In this problem you'll compare exact and linearized analyses of a Hohmann transfer.

a) Assume a "reference" satellite is in a circular orbit of radius ρ about a central body with gravitational parameter μ. What is the mean motion n?

b) Assume a "chaser" spacecraft is in a higher circular orbit of radius ($\rho + h$). The chaser wants to make a Hohmann transfer down to the reference orbit and meet the reference spacecraft at the end of the transfer. What is the transfer time? (Use an exact analysis of the Hohmann transfer.) You should have an expression containing μ, ρ, and h.

c) Because the transfer orbit has a slightly longer period than the reference orbit, at the start of the transfer, the chaser must be "ahead" of the reference by some "phase angle" as measured from the center of the central body. What is the value of this phase angle in [radians]? Your answer should contain ρ and h .

d) Now consider a linearized analysis described by the Clohessy-Wiltshire equations. The reference spacecraft is still at orbital radius ρ. Assume that just after the transfer begins, the chaser has:

$$\begin{bmatrix} x_0 \\ y_0 \\ \dot{x}_{+0} \\ \dot{y}_{+0} \end{bmatrix} = \begin{bmatrix} h \\ ??? \\ 0 \\ -1.75nh \end{bmatrix}$$

and that the transfer takes $t_f = \frac{\pi}{n}$ to complete. Show that this is indeed a Hohmann transfer (i.e. that $\dot{x}(0) = 0$ and $\dot{x}(t_f) = 0$).

e) What must be the starting value of $y(0)$ for the chaser to end up at $y(t_f) = 0$? Show that $x(t_f) = 0$ (i.e. the transfer really does end up at the reference spacecraft).

f) Sketch the chaser's trajectory in the x-y plane. (It's fine if you don't want to use MATLAB. Just make sure you sketch the correct directions of departure from the starting point and approach to the reference.)

g) How long does the transfer take? Express your answer in a way that allows comparison with part b.

h) To find the phase angle required, write $\frac{y(0)}{\rho}$ in [radians]. Express your answer in a way that allows comparison with part c.

i) Now assume some real numbers for a reference spacecraft in LEO:
ρ = 6,378 [km] + 250 [km] , h = 10 [km] , and μ_{Earth} = 398,600 [km^3/sec^2]. Compare the transfer times using the exact and linearized (CW) methods. Do the same thing with the phase angles. Comment on the usefulness of the CW equations in this case.

2.4) (32 points) In this problem you'll compare exact and linearized analyses of a Hohmann transfer.

a) Assume a "reference" satellite is in a circular orbit of radius ρ about a central body with gravitational parameter μ. What is the mean motion n?

b) Assume a "chaser" spacecraft is in a higher circular orbit of radius ($\rho + h$). The chaser wants to make a Hohmann transfer down to the reference orbit and meet the reference spacecraft at the end of the transfer. What is the transfer time? (Use an exact analysis of the Hohmann transfer.) You should have an expression containing μ, ρ, and h.

a) $n = \sqrt{\frac{\mu}{\rho^3}}$

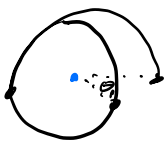
b) 

$a = \frac{2\rho+h}{2}$

$T = 2\pi\sqrt{\frac{a^3}{\mu}} = 2\pi\sqrt{\frac{(2\rho+h)^3}{8\mu}}$

half orbit: $t = \frac{T}{2}$

$$t = \pi\sqrt{\frac{(2\rho+h)^3}{8\mu}}$$



c) Because the transfer orbit has a slightly longer period than the reference orbit, at the start of the transfer, the chaser must be "ahead" of the reference by some "phase angle" as measured from the center of the central body. What is the value of this phase angle in [radians]? Your answer should contain ρ and h.

d) Now consider a linearized analysis described by the Clohessy-Wiltshire equations. The reference spacecraft is still at orbital radius ρ. Assume that just after the transfer begins, the chaser has:

$$\begin{bmatrix} x_0 \\ y_0 \\ \dot{x}_{+0} \\ \dot{y}_{+0} \end{bmatrix} = \begin{bmatrix} h \\ ??? \\ 0 \\ -1.75nh \end{bmatrix}$$

and that the transfer takes $t_f = \frac{\pi}{n}$ to complete. Show that this is indeed a Hohmann transfer (i.e. that $\dot{x}(0) = 0$ and $\dot{x}(t_f) = 0$).

e) What must be the starting value of $y(0)$ for the chaser to end up at $y(t_f) = 0$? Show that $x(t_f) = 0$ (i.e. the transfer really does end up at the reference spacecraft).

f) Sketch the chaser's trajectory in the x-y plane. (It's fine if you don't want to use MATLAB. Just make sure you sketch the correct directions of departure from the starting point and approach to the reference.)

g) How long does the transfer take? Express your answer in a way that allows comparison with part b.

h) To find the phase angle required, write $\frac{y(0)}{\rho}$ in [radians]. Express your answer in a way that allows comparison with part c.

i) Now assume some real numbers for a reference spacecraft in LEO:
$\rho = 6{,}378$ [km] + 250 [km] , $h = 10$ [km] , and $\mu_{Earth} = 398{,}600$ [km^3/sec^2]. Compare the transfer times using the exact and linearized (CW) methods. Do the same thing with the phase angles. Comment on the usefulness of the CW equations in this case.

c) $\theta_c = \pi$ $\quad \theta_t = \theta_p + \pi = n_t t = \sqrt{\frac{\mu}{\rho^3}}\,\pi\sqrt{\frac{(2\rho+h)^3}{8\mu}}$

$\theta_t = \pi\sqrt{\frac{(2\rho+h)^3}{8\rho^3}}$ $\rightarrow$ $\theta_p = \theta_t - \pi = \pi\sqrt{\frac{(2\rho+h)^3}{8\rho^3}} - \pi$

$$\theta_p = \left(\sqrt{\frac{(2\rho+h)^3}{8\rho^3}} - 1\right)\pi$$

d) $\dot{x}(t) = 3n\sin(nt)\,x_0 + \cos(nt)\,\dot{x}_0 + 2\sin(nt)\,\dot{y}_0$

$\dot{x}(0) = 3n\sin(0)\,x_0 + \cos(0)\,\dot{x}_0 + 2\sin(0)\,\dot{y}_0$

$\dot{x}(0) = 0$

$\dot{x}(t_f) = 3n\sin(nt_f)\,x_0 + \cos(nt_f)\,\dot{x}_0 + 2\sin(nt_f)\,\dot{y}_0$

$= 3n(0)\,x_0 + 0 + 2(0)\,\dot{y}_0$ $\quad \dot{x}(t_f) = 0$

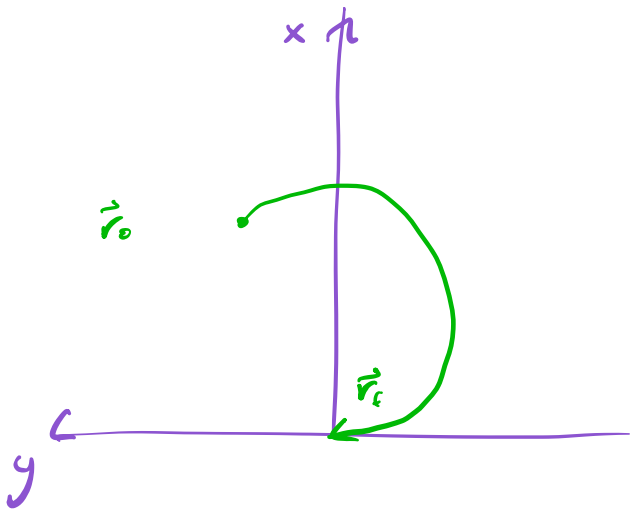
e) $y(t) = 6(\sin(nt) - nt)\,x_0 + y_0 + \frac{2}{n}(\cos(nt) - 1)\,\dot{x}_0 + \left(\frac{4}{n}\sin(nt) - \frac{3}{n}nt\right)\dot{y}_0$

$y(t_f) = 6(\sin(\pi) - \pi)\,x_0 + y_0 + \left(\frac{4}{n}\sin(\pi) - \frac{3}{n}\pi\right)\dot{y}_0$ $\quad -1.75nh$

$0 = -6\pi h + y_0 - \frac{3}{n}\pi(1.75nh) = -6\pi h + \frac{21}{4}\pi h + y_0 = y_0 - \frac{3}{4}\pi h$ $\quad \frac{7}{4}$

$$y_0 = \frac{3}{4}\pi h$$

f)



g) $x(t) = (4 - 3\cos(nt))x_0 + \frac{1}{n}\sin(nt)\dot{y}_0 + \frac{2}{n}(1-\cos(nt))\dot{y}_0$

$x(t_f) = (4 - 3\cos(nt_f))x_0 + \frac{2}{n}(1-\cos(nt_f))\dot{y}_0$

$= 4h - 3h\cos(nt_f) + (1-\cos(nt_f))(-\frac{3}{2}h)$

$= 4h - 3h\cos(nt_f) - \frac{3}{2}h + \frac{3}{2}h\cos(nt_f) = \frac{1}{2}h + \frac{1}{2}h\cos(nt_f) = 0$

$\cos(nt_f) = -1 \quad \therefore \quad nt_f = \pi \rightarrow t_f = \frac{\pi}{n} = \pi\sqrt{\frac{a^3}{\mu}}$

$$\boxed{t_f = \pi\sqrt{\frac{a^3}{\mu}}}$$

h) $\tan\theta_p = \frac{y_0}{\rho} \approx \theta_p = \frac{\frac{3}{4}\pi h}{\rho} = \frac{3\pi}{4}\cdot\frac{h}{\rho} = \pi\left(\frac{3h}{4\rho}\right)$

$$\boxed{\theta_p = \pi\left(\frac{3h}{4\rho}\right)}$$

i) $\rho = 6778 + 250 = 6628$ km $\qquad h = 10$ km $\qquad \mu = 398600 \frac{km^3}{s^2}$

```
Transfer time:
        Exact: 2688.105s
        HCW: 2685.066s (0.113% error)
Phase angle:
        Exact: 0.20372º
        HCW: 0.20368º (0.019% error)
```

Very close for $h \ll \rho$

AA 279B Homework 1

Problem 1

In [1]:
```
%load_ext autoreload
%autoreload 2
```

In [2]:
```
import numpy as np
from numpy.linalg import norm
from scipy.integrate import solve_ivp
from scipy.optimize import root_scalar
import matplotlib.pyplot as plt
from IPython.display import display, Latex
import plotly.graph_objects as go
from tqdm import tqdm
from dataclasses import dataclass
```

In [3]:
```
DAY_TO_SEC = 24 * 60 * 60
MIN_TO_SEC = 60

# km3/m2
MU_EARTH = 398600
MU_MOON = 4903

# km
r_EARTH_MOON = 384400
R_EARTH = 6378.0
R_MOON = 1738.0

# Barycenter
r_bc_earth = MU_MOON / (MU_MOON + MU_EARTH) * r_EARTH_MOON
r_bc_moon = MU_EARTH / (MU_MOON + MU_EARTH) * r_EARTH_MOON

# w_s
w_earth_moon = np.sqrt( (MU_EARTH + MU_MOON) / r_EARTH_MOON**3)
```

L2 (part a)

In [4]:
```
def earth_moon_two_body(x):
    return (MU_EARTH / (x + r_bc_earth)**2 +
            MU_MOON / (x - r_bc_moon)**2 -
            w_earth_moon**2 * x
            )

r_L2_x = root_scalar(
    earth_moon_two_body,
    bracket=[r_bc_moon + 0.001, r_bc_moon*2]).root
```

```
r_L2 = np.array([r_L2_x, 0, 0])
print(f"L2: {r_L2_x}km -> {r_L2}")
```

```
L2: 444244.96544722747km -> [444244.96544723      0.              0.
]
```

Sim (part b)

In [5]:
```
def synodic_dynamics(state, mu1, mu2, R1, R2, w_s):
    """mu1/R1 for larger body"""
    assert mu1 > mu2
    assert R1 < R2

    x, y, z, x_dot, y_dot, z_dot = state

    r_1_sat = np.sqrt((x + R1)**2 + y**2 + z**2)
    r_2_sat = np.sqrt((x - R2)**2 + y**2 + z**2)

    x_ddot = (
        -mu1 * (x + R1) / (r_1_sat**3)
        -mu2 * (x - R2) / (r_2_sat**3)
        + 2 * w_s * y_dot
        + w_s**2 * x)

    y_ddot = (
        -mu1 * y / (r_1_sat**3)
        -mu2 * y / (r_2_sat**3)
        - 2 * w_s * x_dot
        + w_s**2 * y)

    z_ddot = (
        -mu1 * z / (r_1_sat**3)
        -mu2 * z / (r_2_sat**3))

    return np.array([x_dot, y_dot, z_dot, x_ddot, y_ddot, z_ddot])
```

In [6]:
```
# Wrapper
def earth_moon_synodic_dynamics(t, state):
    del t
    return synodic_dynamics(state, MU_EARTH, MU_MOON, r_bc_earth, r_bc_moon,
```

In [7]:
```
# Initial conditions

sim_length = 4.0 * DAY_TO_SEC
t_arr_4_day = np.linspace(0, sim_length, 100000)

convergence_bound = 1 # km
```

In [8]:
```
# VERY SPECIFIC to Earth
def earth_inputs_to_state(phi_inj, v_inj, altitude = 200):
    state0 = np.zeros(6)

    rho_earth_sat = altitude + R_EARTH
    # x,y
    state0[0] = -rho_earth_sat*np.cos(phi_inj) - r_bc_earth
```

```
    state0[1] = -rho_earth_sat*np.sin(phi_inj)

    # vx,vy
    state0[3] =  v_inj * np.sin(phi_inj)
    state0[4] = -v_inj * np.cos(phi_inj)

    return state0
```

In [9]:
```
# Shoot.
def simulate_earth_moon(t_arr, state0):
    t_span = (t_arr[0], t_arr[-1])
    return solve_ivp(earth_moon_synodic_dynamics, t_span, state0, method='LS

def jacobian(phi_inj, v_inj, t_arr, delta = 0.001):
    """2 inputs to 3 outputs (r) -> (1,2) matrix"""

    # phi variation
    phi_sim_upper = simulate_earth_moon(t_arr, earth_inputs_to_state(phi_inj
    phi_sim_lower = simulate_earth_moon(t_arr, earth_inputs_to_state(phi_inj

    dr_dphi = (phi_sim_upper[0:3,-1]  - phi_sim_lower[0:3,-1] ) / (2*delta)

    # vel variation
    vel_sim_upper = simulate_earth_moon(t_arr, earth_inputs_to_state(phi_inj
    vel_sim_lower = simulate_earth_moon(t_arr, earth_inputs_to_state(phi_inj

    dr_dv = (vel_sim_upper[0:3,-1] - vel_sim_lower[0:3,-1]) / (2*delta)

    return np.hstack((dr_dphi.reshape((3,-1)), dr_dv.reshape((3,-1))))
    # return np.concat((dr_dphi, dr_dv)).reshape(6,-1)
```

In [10]:
```
"""
Pseudocode:
- Seed loosely around previous trajectory
- Keep N trajectories
"""

def find_initial_conditions(t_arr, phi0_deg = 45, v_inj_km_s = 11, verbose =

    phi_arr = [np.deg2rad(phi0_deg)]
    v_inj_arr = [v_inj_km_s]
    sim_arr = []

    iter = 10
    for i in range(iter):
        sim = simulate_earth_moon(t_arr, earth_inputs_to_state(phi_arr[-1],
        sim_arr.append(sim)

        dr = sim[0:3,-1] - r_L2
        dr_norm = norm(dr)

        if verbose:
            print(f"dr: {dr_norm:.5f}km")

        if norm(dr) < convergence_bound:
```

```
            print(f"\nConverged with:\n\tdr = {dr_norm:.5f} km")
            break

        J = jacobian(phi_arr[-1], v_inj_arr[-1], t_arr)

        new = [phi_arr[-1], v_inj_arr[-1]] - np.linalg.pinv(J) @ dr

        phi_arr.append(new[0])
        v_inj_arr.append(new[1])

    return sim_arr, phi_arr, v_inj_arr
```

In [11]:
```
sims, phis, v_injs = find_initial_conditions(t_arr_4_day, phi0_deg = 45, v_i
first_sim = sims[0]
sim = sims[-1]
phi = np.rad2deg(phis[-1])
v_inj = v_injs[-1]
```

```
dr: 262888.69755km
dr: 111685.42353km
dr: 39045.19353km
dr: 1936.39399km
dr: 5.75182km
dr: 0.00001km

Converged with:
        dr = 0.00001 km
```

Plotting (part c)

In [12]:
```
fig, ax = plt.subplots(figsize=(20, 8))
ax.set_title(f'Final shooting trajectory to L2:   phi = {phi:.1f}\u00b0,  v_

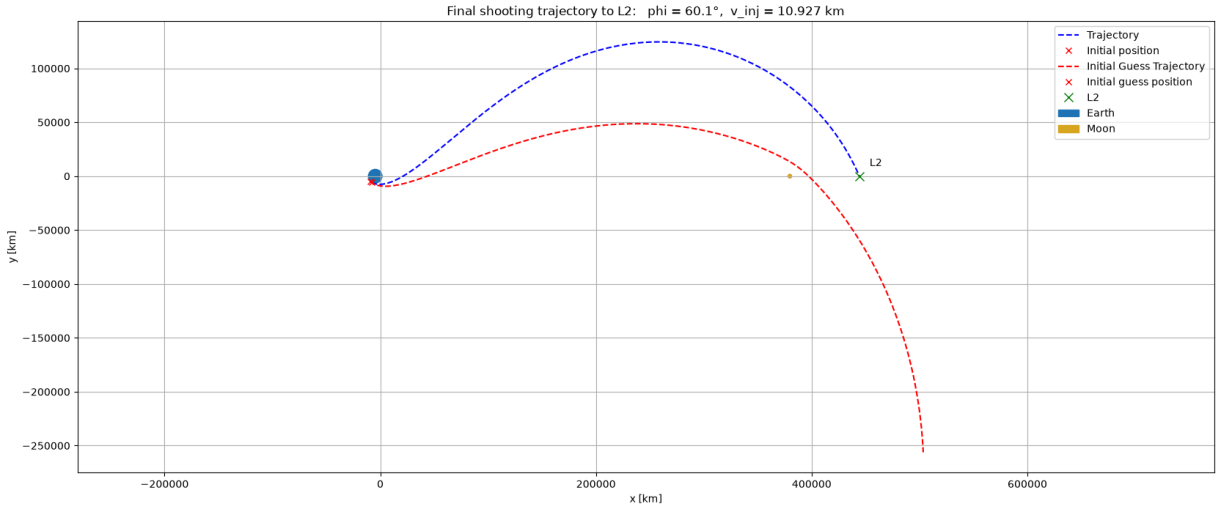
# Trajectory
ax.plot(sim[0], sim[1], 'b--', label='Trajectory')
ax.plot(sim[0,0], sim[1,1], 'rx', label='Initial position')
ax.plot(first_sim[0], first_sim[1], 'r--', label='Initial Guess Trajectory')
ax.plot(first_sim[0,0], first_sim[1,1], 'rx', label='Initial guess position'

# Bodies
# ax.plot(-r_bc_earth, 0, "bo", markersize=5, label="Earth")
ax.plot(r_L2_x, 0, "x", color="green", markersize=8, label="L2")
plt.annotate("L2", xy=(r_L2_x,0),textcoords="offset points", xytext=(10,10))

ax.add_patch(plt.Circle((-r_bc_earth,0), R_EARTH, color='tab:blue', fill=Tru
ax.add_patch(plt.Circle((r_bc_moon,0), R_MOON, color='goldenrod', fill=True,

ax.set_xlabel('x [km]')
ax.set_ylabel('y [km]')
ax.set_aspect('equal', adjustable='datalim')

ax.grid(True)
ax.legend()
plt.show()
```



In [13]:
```
# Around Earth

fig, ax = plt.subplots(figsize=(20, 10))
ax.set_title(f'Final shooting trajectory to L2:   phi = {phi:.1f}\u00b0,  v_

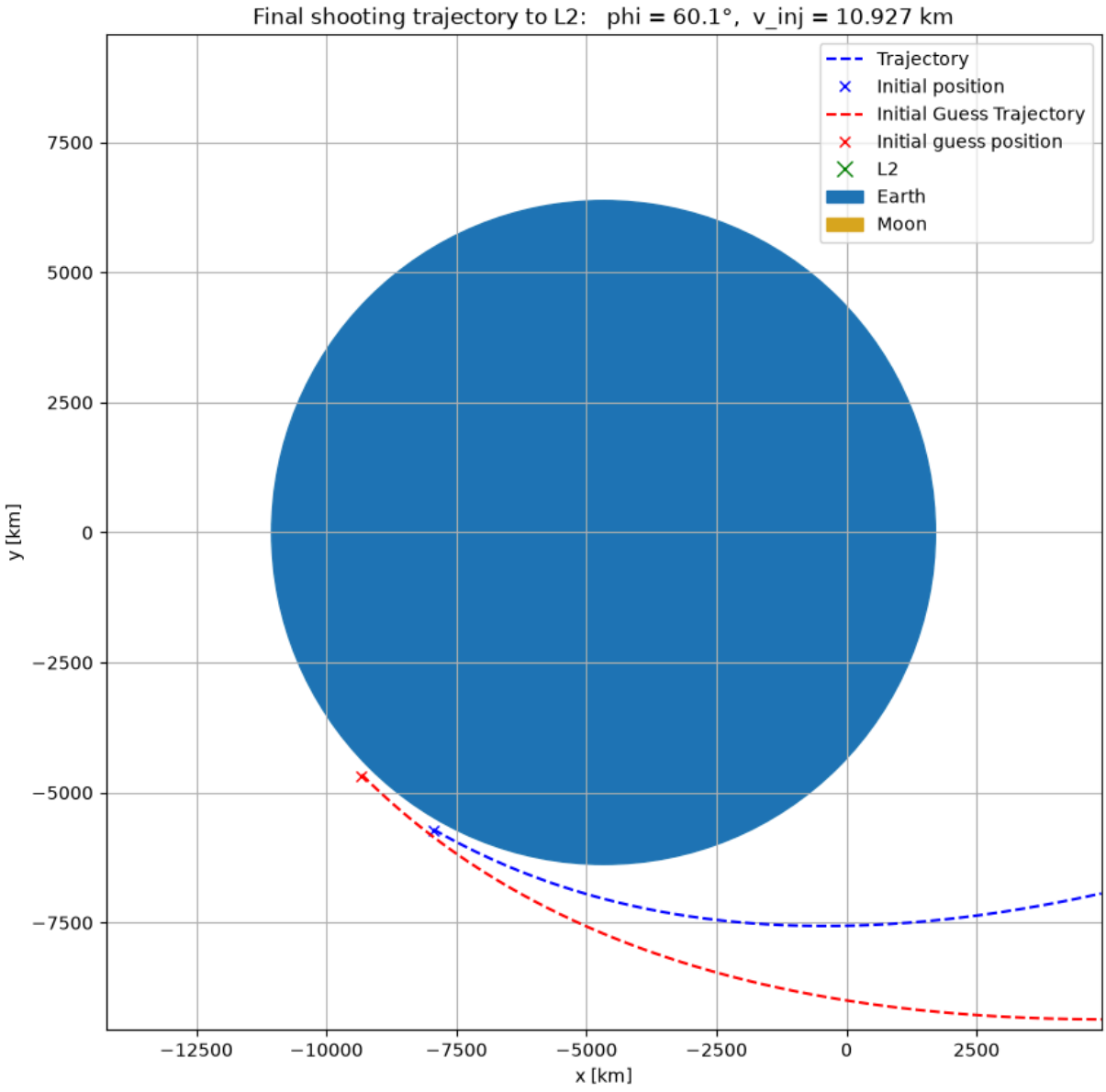
# Trajectory
ax.plot(sim[0], sim[1], 'b--', label='Trajectory')
ax.plot(sim[0,0], sim[1,1], 'bx', label='Initial position')
ax.plot(first_sim[0], first_sim[1], 'r--', label='Initial Guess Trajectory')
ax.plot(first_sim[0,0], first_sim[1,1], 'rx', label='Initial guess position'

# Bodies
# ax.plot(-r_bc_earth, 0, "bo", markersize=5, label="Earth")
ax.plot(r_L2_x, 0, "x", color="green", markersize=8, label="L2")
plt.annotate("L2", xy=(r_L2_x,0),textcoords="offset points", xytext=(10,10))

ax.add_patch(plt.Circle((-r_bc_earth,0), R_EARTH, color='tab:blue', fill=Tru
ax.add_patch(plt.Circle((r_bc_moon,0), R_MOON, color='goldenrod', fill=True,

ax.set_xlabel('x [km]')
ax.set_ylabel('y [km]')
ax.set_aspect('equal')

ax.grid(True)
ax.legend()
ax.set_xlim(-r_bc_earth - R_EARTH*1.5, -r_bc_earth + R_EARTH*1.5)
ax.set_ylim(-R_EARTH*1.5, R_EARTH*1.5)
plt.show()
```



v_retro (part d)

In [14]:
```
dv_retro_L2 = norm(sim[3:6,-1])
print(f"dV_L2: {dv_retro_L2}")
```

```
dV_L2: 1.2306210650865435
```

Prop estimate (part e)

In [15]:
```
def calc_dv_tot(sim):
    dv_retro_L2 = norm(sim[3:6,-1])
    rho = R_EARTH + 200
    v_orbital = np.sqrt(MU_EARTH / rho - w_earth_moon * rho)
    dv_tot = v_inj - v_orbital + dv_retro_L2
    print(f"dv_tot: {dv_tot} km/s")

dv_tot = calc_dv_tot(sim)
```

```
dv_tot: 4.374548381075959 km/s
```

Optimizing fuel

In [16]:
```
# N = 100
N = 0
day_arr = np.linspace(6,7,N)
t_arr_days = [np.linspace(0, day * DAY_TO_SEC, 100000) for day in day_arr]
dv_tot_arr = np.zeros_like(t_arr_days)

for i in tqdm(range(N)):
    sims, phis, v_injs = find_initial_conditions(t_arr_days[i], phi0_deg = 4
    sim = sims[-1]
    phi = np.rad2deg(phis[-1])
    v_inj = v_injs[-1]

    dv_tot_arr[i] = calc_dv_tot(sim)
```

```
0it [00:00, ?it/s]
0it [00:00, ?it/s]
```

Problem 2

part a

In [17]:
```
r_I = np.array([343777.2401, 0.0, -27449.52657])
v_I = np.array([0.0, -0.2769119497, 0.0])
```

In [18]:
```
t_arr_30 = np.linspace(0, 30 * DAY_TO_SEC, 100000)
sim_30 = simulate_earth_moon(t_arr_30, np.concatenate((r_I, v_I))).y
```

In [19]:
```
def plot_3d_(r, markers = None):

    fig = go.Figure()

    # Satellite orbit
    fig.add_trace(go.Scatter3d(
        x=r[0],
        y=r[1],
        z=r[2],
        mode="lines",
        name="Satellite orbit",
        line=dict(color="royalblue", width=5)
    ))

    # Satellite's final/current position
    fig.add_trace(go.Scatter3d(
        x=[r[0, -1]],
        y=[r[1, -1]],
        z=[r[2, -1]],
        mode="markers",
        name="Satellite",
```

```
        marker=dict(size=6, color="blue")
    ))

    # L2 point

    markers = markers if markers else []
    for marker in markers:
        fig.add_trace(go.Scatter3d(
            x=[marker[0]],
            y=[marker[1]],
            z=[marker[2]],
            mode="markers+text",
            name="L2 point",
            text=["L2"],
            textposition="top center",
            marker=dict(size=8, color="red", symbol="diamond")
        ))

    fig.update_layout(
        title="3D Satellite Orbit and L2 Point",
        scene=dict(
            xaxis_title="x",
            yaxis_title="y",
            zaxis_title="z",
            aspectmode="data"
        ),
        template="plotly_white",
        legend=dict(x=0.02, y=0.98)
    )
    fig.update_layout(height=900)

    fig.show()
```

In [20]:
```
plot_3d_(sim_30[:3, :], [r_L2])
```

In [21]:
```
def estimate_periods(sim):
    y_halo = sim[2, :]
    y_crossings_ascending = np.where((y_halo[:-1] <= 0) & (y_halo[1:] > 0))[
    t_crossings = t_arr_30[y_crossings_ascending]
    periods = np.array([t_crossings[i+1] - t_crossings[i] for i in range(ler

    return periods


periods = estimate_periods(sim_30)
print(f"Periods: {periods}s ({periods / DAY_TO_SEC} day)")
```

```
Periods: [1044975.2497525  1086914.22914229]s ([12.09462095 12.5800258 ] da
y)
```

part 2

In [22]:
```
r_I_2 = np.array([343117.419, 0.0, 76029.48372])

v_I_2 = np.array([0.0, 0.2045850249, 0.0])
```

```
t_arr_30_2 = np.linspace(0, 30 * DAY_TO_SEC, 100000)
sim_30_2 = simulate_earth_moon(t_arr_30_2, np.concatenate((r_I_2, v_I_2))).y

plot_3d_(sim_30_2[:3, :])
```

In [23]:
```
periods = estimate_periods(sim_30_2)
print(f"Periods: {periods}s ({periods / DAY_TO_SEC} day)")
```

```
Periods: [745829.53829538 745803.61803618]s ([8.63228632 8.63198632] day)
```

Problem 3

In [24]:
```
target_alt = 760  # km
chaser_r0 = np.array([25, -75])  # 2D [x,y,vx,vy]
chaser_v0 = np.array([0, 0])  # 2D [x,y,vx,vy]
intercept_time = 10 * MIN_TO_SEC

r_goal = np.array([0,0])
v_goal = np.array([0,0])
```

In [25]:
```
mean_motion = np.sqrt(MU_EARTH / (R_EARTH + target_alt)**3)
```

part a

In [26]:
```
@dataclass
class Phi:
    rr: np.ndarray
    rv: np.ndarray
    vv: np.ndarray
    vr: np.ndarray


def calc_Phi(t, n):
    nt = n * t
    Phi_rr = np.array([[4 - 3*np.cos(nt), 0], [6 * (np.sin(nt) - nt), 1]])
    Phi_rv = np.array([
        [1/mean_motion * np.sin(nt), 2/mean_motion * (1 - np.cos(nt))],
        [2/mean_motion * (np.cos(nt) - 1), 4/mean_motion * np.sin(nt) - 3/me
        ])
    Phi_vr = np.array([[3*mean_motion*np.sin(nt), 0], [6*mean_motion*(np.cos
    Phi_vv = np.array([
        [np.cos(nt), 2*np.sin(nt)],
        [-2*np.sin(nt), 4*np.cos(nt) - 3]
        ])

    return Phi(rr=Phi_rr, rv=Phi_rv, vv=Phi_vv, vr=Phi_vr)

phi = calc_Phi(intercept_time, mean_motion)

dv_start = np.linalg.pinv(phi.rv) @ (r_goal - phi.rr @ chaser_r0) - chaser_v
print(f"dv_start: {norm(dv_start):.3f} km/s   ({dv_start})")
```

```
dv_start: 0.147 km/s   ([-0.12292895  0.08140932])
```

part b

In [27]:
```
dv_end = v_goal - (phi.vr @ chaser_r0 + phi.vv @ (chaser_v0 + dv_start))
print(f"dv_end: {norm(dv_end):.3f} km/s   ({dv_end})")

dv_tot = norm(dv_end) + norm(dv_start)

print(f"\ndv_tot: {dv_tot:.3f} km/s")
```

```
dv_end: 0.140 km/s   ([-0.04235379 -0.13375416])

dv_tot: 0.288 km/s
```

In [28]:
```
t_hcw = np.arange(0, intercept_time + 1)

def calc_r_hcw(t, n, r0, v0):
    nt = n * t
    Phi_rr = np.array([[4 - 3*np.cos(nt), 0], [6 * (np.sin(nt) - nt), 1]])
    Phi_rv = np.array([
        [1/n * np.sin(nt), 2/n * (1 - np.cos(nt))],
        [2/n * (np.cos(nt) - 1), 4/n * np.sin(nt) - 3/n * nt]
        ])
    return Phi_rr @ r0 + Phi_rv @ v0

r_hcw = np.array([calc_r_hcw(t_i, mean_motion, chaser_r0, dv_start) for t_i
x_hcw = r_hcw[:,0]
y_hcw = r_hcw[:,1]
```

In [29]:
```
fig, ax = plt.subplots(figsize=(20, 8))
ax.set_title('Chaser Spacecraft Trajectory over 10 minutes')

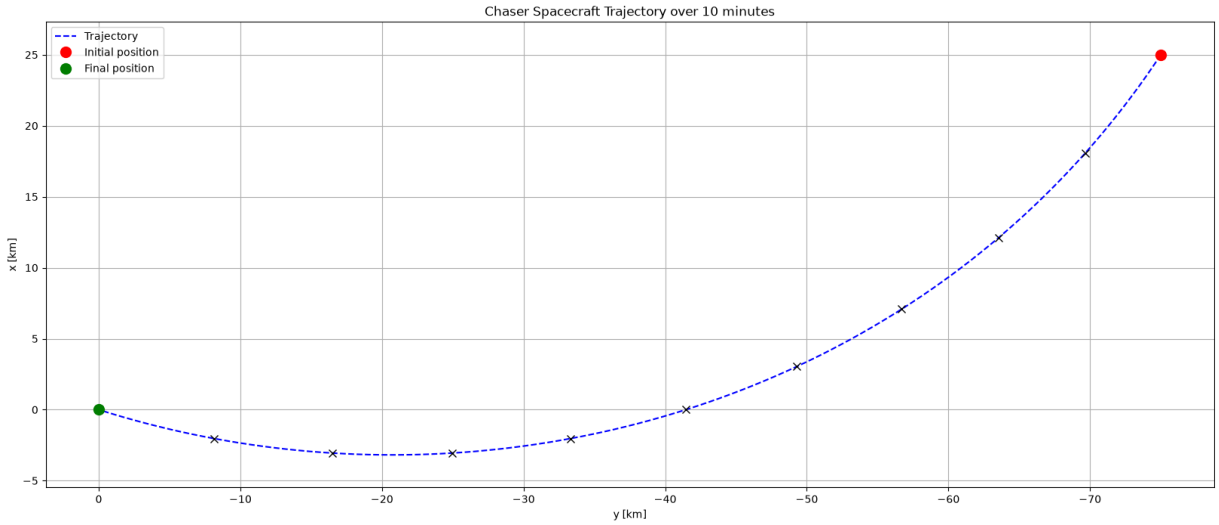
# Trajectory
ax.plot(y_hcw, x_hcw, 'b--', label='Trajectory')


for i in range(0, len(x_hcw), MIN_TO_SEC):
    ax.plot(y_hcw[i], x_hcw[i], 'kx', markersize=7)

ax.plot(y_hcw[0], x_hcw[0], 'ro', markersize=10, label='Initial position')
ax.plot(y_hcw[-1], x_hcw[-1], 'go', markersize=10, label='Final position')

ax.set_xlabel('y [km]')
ax.set_ylabel('x [km]')
ax.set_aspect('equal', adjustable='datalim')
ax.set_xinverted(True)

ax.grid(True)
ax.legend()
plt.show()
```



part c

In [30]:
```
tf_arr = np.arange(1, 121)

def calc_v_tot(intercept_time_s,n,r0,v0,r_goal=np.zeros(2),v_goal=np.zeros(2
    phi = calc_Phi(intercept_time_s, n)

    dv_start = np.linalg.pinv(phi.rv) @ (r_goal - phi.rr @ r0) - v0
    dv_end = v_goal - (phi.vr @ r0 + phi.vv @ (v0 + dv_start))

    return norm(dv_start) + norm(dv_end)

dv_arr = np.array([calc_v_tot(tf * MIN_TO_SEC, mean_motion, chaser_r0, chase

dv_arr[dv_arr > 5] = np.nan
```

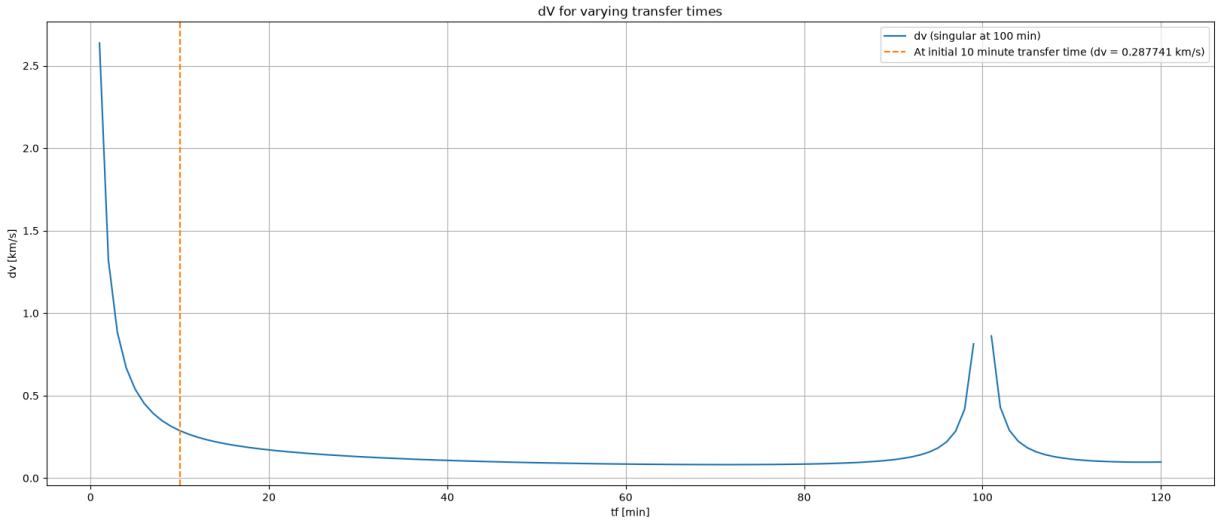
In [31]:
```
fig, ax = plt.subplots(figsize=(20, 8))
ax.set_title('dV for varying transfer times')

# Trajectory
ax.plot(tf_arr, dv_arr, label="dv (singular at 100 min)")
ax.axvline(10, linestyle="--", color="tab:orange", label=f'At initial 10 mir

ax.set_ylabel('dv [km/s]')
ax.set_xlabel('tf [min]')

ax.grid(True)
ax.legend()

plt.show()
```



part d

In [32]:
```
dv_min = np.nanmin(dv_arr)
t_dv_min = tf_arr[np.nanargmin(dv_arr)]

print(f"Minimum:\n\tdv = {dv_min:.4f} km/s\nat\n\tt = {t_dv_min} min")
```

```
Minimum:
	dv = 0.0816 km/s
at
	t = 71 min
```

In [33]:
```
def calc_r_hcw(t, n, r0, v0):
    nt = n * t
    Phi_rr = np.array([[4 - 3*np.cos(nt), 0], [6 * (np.sin(nt) - nt), 1]])
    Phi_rv = np.array([
        [1/n * np.sin(nt), 2/n * (1 - np.cos(nt))],
        [2/n * (np.cos(nt) - 1), 4/n * np.sin(nt) - 3/n * nt]
        ])
    return Phi_rr @ r0 + Phi_rv @ v0

t_hcw = np.arange(0, t_dv_min*MIN_TO_SEC + 1)

r_hcw = np.array([calc_r_hcw(t_i, mean_motion, chaser_r0, dv_start) for t_i
x_hcw = r_hcw[:,0]
y_hcw = r_hcw[:,1]
```

In [34]:
```
fig, ax = plt.subplots(figsize=(20, 8))
ax.set_title(f"Chaser Spacecraft Trajectory over {t_dv_min} minutes with opt

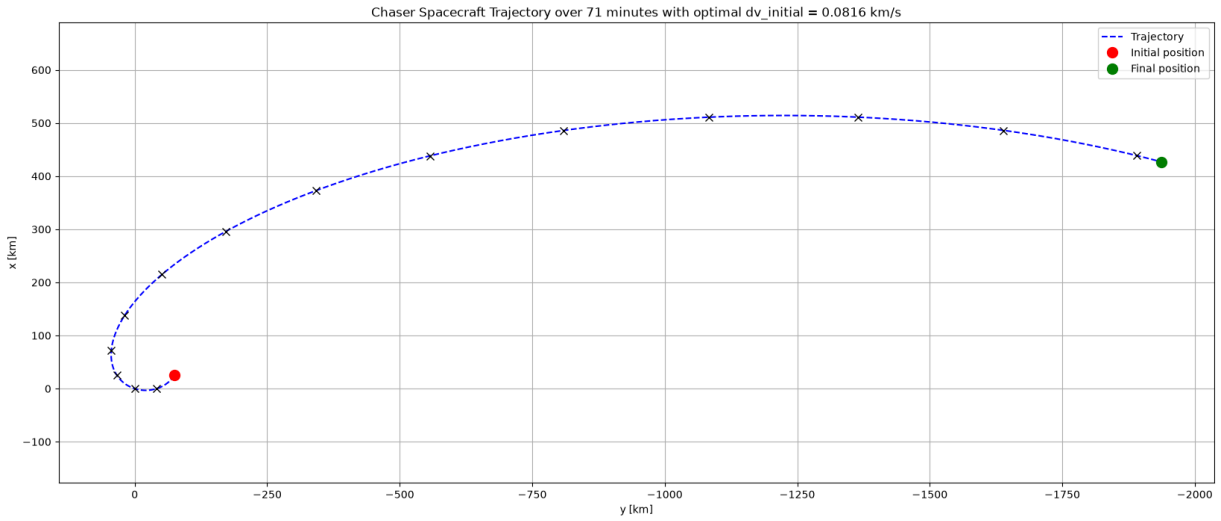
# Trajectory
ax.plot(y_hcw, x_hcw, 'b--', label='Trajectory')

for i in range(0, len(x_hcw), 5*MIN_TO_SEC):
    ax.plot(y_hcw[i], x_hcw[i], 'kx', markersize=7)

ax.plot(y_hcw[0], x_hcw[0], 'ro', markersize=10, label='Initial position')
ax.plot(y_hcw[-1], x_hcw[-1], 'go', markersize=10, label='Final position')
```

```
ax.set_xlabel('y [km]')
ax.set_ylabel('x [km]')
ax.set_aspect('equal', adjustable='datalim')
ax.set_xinverted(True)

ax.grid(True)
ax.legend()
plt.show()
```



Problem 4

In [35]:
```
rho = R_EARTH + 250
h = 10
```

In [36]:
```
t_exact = np.pi * np.sqrt( (2*rho + h)**3 / 8 / MU_EARTH)
theta_exact = ( np.sqrt( (2*rho + h)**3 / 8 / (rho**3) ) - 1) * np.pi

t_hcw = np.pi * np.sqrt( rho**3 / MU_EARTH)
theta_hcw = ( 3 * h / 4 / rho) * np.pi

t_err = abs(t_hcw - t_exact) / t_exact
theta_err = abs(theta_hcw - theta_exact) / theta_exact
```

In [37]:
```
print("Transfer time:")
print(f"\tExact: {t_exact:.3f}s")
print(f"\tHCW: {t_hcw:.3f}s ({t_err*100:.3f}% error)")
print("Phase angle:")
print(f"\tExact: {np.rad2deg(theta_exact):.5f}º")
print(f"\tHCW: {np.rad2deg(theta_hcw):.5f}º ({theta_err*100:.3f}% error)")
```

```
Transfer time:
	Exact: 2688.105s
	HCW: 2685.066s (0.113% error)
Phase angle:
	Exact: 0.20372º
	HCW: 0.20368º (0.019% error)
```